

Model Evaluation

Know If Your Model Is Actually Good

■ EVALUATION & VALIDATION

■ LEARNING OUTCOME

By the end of this PDF you will correctly evaluate any ML model using the right metrics, diagnose model problems, prevent data leakage, and confidently interpret model results for business stakeholders.

■ Skills You Will Gain

- Choose the right metric for classification and regression
- Read and interpret confusion matrix, ROC-AUC, PR curve
- Diagnose overfitting and underfitting
- Prevent data leakage in ML pipelines
- Use SHAP for model explainability
- Report model performance in business language

■ A model with 99% accuracy that predicts fraud might be useless — if 99% of transactions are not fraud, always guessing 'not fraud' gets 99%. Knowing evaluation metrics = knowing truth.

SECTION 1

Classification Metrics

Metric	Formula	When to Use
Accuracy	$(TP+TN) / \text{Total}$	Balanced classes only — misleading for imbalanced
Precision	$TP / (TP+FP)$	Minimize false positives — spam filter, cancer screening
Recall (Sensitivity)	$TP / (TP+FN)$	Minimize false negatives — fraud detection, disease diagnosis
F1 Score	$2 \times (P \times R) / (P+R)$	Balance precision & recall — imbalanced datasets
AUC-ROC	Area under ROC curve [0,1]	Overall discrimination ability — model ranking
Log Loss	$-(y \cdot \log(p) + (1-y) \cdot \log(1-p))$	Probabilistic predictions — calibration quality
MCC	Balanced metric for all 4 confusion values	Best single metric for imbalanced binary class

SECTION 2

Confusion Matrix Deep Dive

```
from sklearn.metrics import (confusion_matrix, classification_report, roc_auc_score, roc_curve,
precision_recall_curve, f1_score) import seaborn as sns # Confusion matrix cm =
confusion_matrix(y_test, y_pred) sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Pred 0', 'Pred 1'], yticklabels=['True 0', 'True 1']) plt.title('Confusion Matrix');
plt.show() # Full classification report print(classification_report(y_test, y_pred,
target_names=['No Churn', 'Churned'])) # ROC-AUC y_proba = model.predict_proba(X_test)[: ,1] auc =
roc_auc_score(y_test, y_proba) fpr, tpr, _ = roc_curve(y_test, y_proba) plt.plot(fpr, tpr,
label=f'AUC = {auc:.3f}') plt.plot([0,1],[0,1], '--', label='Random') plt.legend();
plt.xlabel('FPR'); plt.ylabel('TPR'); plt.show()
```

SECTION 3

Regression Metrics

Metric	Formula	Interpretation
MAE	$\text{mean}(y - \hat{y})$	Average absolute error — same units as target, robust to outliers
MSE	$\text{mean}((y - \hat{y})^2)$	Penalizes large errors heavily — sensitive to outliers
RMSE	$\sqrt{\text{MSE}}$	Same units as target, penalizes large errors
R ²	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Variance explained [0,1] — 0.85 means 85% of variance captured
MAPE	$\text{mean}(y - \hat{y} / y) \times 100$	Percentage error — interpretable as '±X%'
Adjusted R ²	$1 - (1 - R^2)(n-1)/(n-p-1)$	R ² penalized for number of features — use for model comparison

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
print(f'MAE: {mae:.2f} | RMSE: {rmse:.2f} | R²: {r2:.4f} | MAPE: {mape:.1f}%')
```

SECTION 4

Diagnosing Model Problems

Symptom	Diagnosis	Fix
High train acc, low test acc	Overfitting	More data, regularization, simpler model, dropout
Low train AND test acc	Underfitting	More complex model, more features, less regularization
99% accuracy, useless model	Class imbalance	Use F1/AUC, SMOTE oversampling, class_weight='bal
Model worse after adding data	Data leakage	Find and remove features containing target information
Train/val gap increases over epochs	Overfitting in deep learning	Early stopping, dropout, data augmentation
Model varies wildly between CV folds	High variance	More data, reduce model complexity, increase regulariz

SECTION 5

Handling Class Imbalance

```
# Method 1: Class weights (easiest)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(class_weight='balanced') # auto-weights
# Method 2: SMOTE – create synthetic minority samples
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)
# Method 3: Adjust decision threshold
y_proba = model.predict_proba(X_test)[:,-1]
threshold = 0.3 # default 0.5 – lower = catch more positives
y_pred_custom = (y_proba >= threshold).astype(int)
# Always use F1/AUC instead of accuracy for imbalanced!
auc = roc_auc_score(y_test, y_proba)
f1 = f1_score(y_test, y_pred_custom)
print(f'AUC: {auc:.4f} | F1: {f1:.4f}')
```

SECTION 6

Model Explainability with SHAP

```
pip install shap
import shap # Tree SHAP (Random Forest, XGBoost, LightGBM)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
# Summary plot – feature importance + direction
shap.summary_plot(shap_values, X_test, plot_type='bar')
# importance
shap.summary_plot(shap_values[1], X_test)
# direction (beeswarm)
# Explain single prediction
shap.waterfall_plot(explainer(X_test)[0])
# why model said yes for row 0
# SHAP tells you: Feature X pushed the prediction UP by 0.12
# while Feature Y pushed it DOWN by 0.08 for this specific prediction
```

PRACTICE QUESTIONS

Q1. Your fraud detection model shows 97% accuracy but catches only 20% of actual frauds. What metric should you look at?

Answer: Recall (sensitivity) — it's critically low at 20%. For fraud detection, missing a fraud (false negative) is much worse than flagging a legitimate transaction.

Q2. What does AUC-ROC = 0.5 mean?

Answer: The model performs no better than random guessing. $AUC=1.0$ is perfect; $AUC=0.5$ is random; $AUC<0.5$ means the model is inverting predictions.

Q3. What is data leakage and give one example?

Answer: When information from the future or test set leaks into training. Example: using 'total account balance at end of year' to predict end-of-year churn — the balance includes the churn decision.

Q4. $R^2 = 0.65$ for a housing price model. Is this good?

Answer: It means 65% of variance in house prices is explained by the model. Whether it's 'good' depends on context — for complex real estate, 0.65 might be acceptable; for simpler tasks it might be too low.

Q5. What does SHAP tell you that regular feature importance doesn't?

Answer: SHAP shows both the magnitude AND direction of each feature's impact on each individual prediction. Regular importance only shows average magnitude across all predictions.

■ Model Evaluation Cheat Sheet	
<code>accuracy_score(y, yp)</code>	Overall accuracy
<code>f1_score(y, yp)</code>	Balanced precision+recall
<code>roc_auc_score(y, yp)</code>	AUC-ROC score
<code>confusion_matrix(y, yp)</code>	4-cell prediction matrix
<code>classification_report(y, yp)</code>	Full P/R/F1 report
<code>mean_absolute_error(y, yp)</code>	MAE – regression
<code>r2_score(y, yp)</code>	R^2 – variance explained
<code>cross_val_score(m, X, y, cv=5)</code>	Cross-validated score
<code>SMOTE().fit_resample(X, y)</code>	Oversample minority
<code>class_weight='balanced'</code>	Auto-weight imbalanced
<code>shap.TreeExplainer(model)</code>	SHAP explainability
<code>shap.summary_plot(sv, X)</code>	Feature importance plot